ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	1 семестр, 2025-2026 гг.

Практическое занятие №6

Использование ORM (GORM). Модели, миграции и связи между таблицами.

Цели

- Понять, что такое ORM и чем удобен GORM.
- Научиться описывать модели Go-структурами и автоматически создавать таблицы (миграции через `AutoMigrate`).
- Освоить базовые связи: **1:N** и **M:N** + выборки с `Preload`.
- Написать короткий REST (2–3 ручки) для проверки результата.

Теоретическая часть: ORM и GORM

1. Что такое ORM

ORM (Object–Relational Mapping) — это технология сопоставления объектно-ориентированных сущностей программы (например, структур в Go) с таблицами реляционной базы данных.

Идея:

- у нас есть **структуры** в коде → ORM «переводит» их в таблицы БД;
- у нас есть **связи между структурами** → ORM «создаёт» внешние ключи;
- мы пишем код «в терминах структур и методов», а ORM формирует SQL-запросы.

Пример:

```
db.Create(&User{Name: "Alice"})
```

ORM сам создаст SQL-запрос:

```
INSERT INTO users (name) VALUES ('Alice');
```

Таким образом, **разработчик думает объектами, а не SQL-строками**.

2. Зачем нужен ORM

Преимущества:

1. **Ускорение разработки** — меньше шаблонного SQL, код компактнее.
2. **Безопасность** — ORM автоматически использует параметризацию (`?` или `$1`) и защищает от SQL-инъекций.
3. **Поддержка миграций** — создание/обновление схемы таблиц прямо из кода.
4. **Кросс-СУБД** — один и тот же код работает с PostgreSQL, MySQL, SQLite.

Недостатки:

- ORM может быть «медленнее» ручного SQL (особенно в больших проектах).
- Для сложных запросов (агрегации, оконные функции) часто приходится писать сырой SQL.

Вывод: ORM идеально подходит для **быстрого старта и стандартных CRUD-операций**.

3. Что такое GORM

GORM — это самая популярная ORM-библиотека на языке Go. Официальный сайт: https://gorm.io.

Основные возможности:

- автоматическая генерация таблиц по структурам (`AutoMigrate`);
- CRUD-операции: `Create`, `First`, `Find`, `Save`, `Delete`;
- фильтрация и сортировка (`Where`, `Order`, `Limit`);
- связи **1:1**, **1:N**, **M:N** (`has one`, `has many`, `many to many`);
- автоматическая подгрузка связанных данных (`Preload`);
- работа с транзакциями;
- хуки (функции, вызываемые до/после операций).

4. Базовые операции в GORM

Предположим, у нас есть модель:

```
type User struct {
  ID    uint
  Name  string
  Email string
}
```

Создание записи

```
db.Create(&User{Name: "Alice", Email: "alice@example.com"})
```

Чтение записи

```
var user User
db.First(&user, 1)              // SELECT * FROM users WHERE id = 1 LIMIT 1;
db.First(&user, "email = ?", "alice@example.com")
```

Обновление записи

```
db.Model(&user).Update("Name", "Alice Cooper")
```

Удаление записи

```
db.Delete(&user)
```

5. Автоматические миграции

GORM умеет создавать и обновлять таблицы на основе структур. Метод `AutoMigrate`:

```
db.AutoMigrate(&User{})
```

- создаст таблицу `users`, если её нет;
- добавит новые поля, если они появились в структуре.

Но `AutoMigrate` не удаляет поля и не меняет типы (это нужно делать вручную или через внешние миграции).

6. Связи в GORM

ORM позволяет работать со связями без ручного SQL.

1:1 (один к одному)

```
type Passport struct {
  ID     uint
  Number string
  UserID uint
  User   User
}
```

1:N (один ко многим)

```
type User struct {
  ID    uint
  Name  string
  Notes []Note // связь: один пользователь → много заметок
}

type Note struct {
  ID     uint
  Title  string
  UserID uint
}
```

M:N (многие ко многим)

```
type Note struct {
  ID    uint
  Title string
```

```
  Tags  []Tag `gorm:"many2many:note_tags;"`
}

type Tag struct {
  ID    uint
  Name  string
  Notes []Note `gorm:"many2many:note_tags;"`
}
```

Загрузка связанных данных

По умолчанию GORM не подгружает связанные записи (экономия ресурсов).

Чтобы загрузить вместе:

```
db.Preload("Notes").First(&user, 1)
```

→ вернёт пользователя + его заметки.

7. Где ORM удобнее, а где нет

ORM лучше всего применять для:

- типовых CRUD-операций (создание, чтение, обновление, удаление);
- простых связей между таблицами;
- быстрой разработки (например, учебные проекты, MVP, стартапы).

Но SQL остаётся важным:

- для сложной аналитики;
- для оптимизации запросов;
- когда нужны хранимые процедуры или специфичные возможности PostgreSQL.

Краткий итог

1. ORM — это «переводчик» между структурами Go и таблицами SQL.
2. GORM — самая удобная ORM в Go.
3. CRUD через GORM пишется в несколько строк кода.
4. Миграции (`AutoMigrate`) позволяют автоматически создавать таблицы.
5. Поддерживаются все типы связей: 1:1, 1:N, M:N.
6. ORM не отменяет SQL, но снимает с разработчика рутину.

Практическая часть

0) Создаём БД

В psql:

```
CREATE DATABASE pz6_gorm;
```

1) Старт проекта и зависимости

```
mkdir pz6-gorm && cd pz6-gorm
go mod init example.com/pz6-gorm
go get gorm.io/gorm gorm.io/driver/postgres github.com/go-chi/chi/v5
```

2) Мини-структура проекта

```
pz6-gorm/
  cmd/server/main.go
  internal/db/postgres.go
  internal/models/models.go
  internal/http/router.go
  internal/http/handlers.go
  go.mod
```

3) Подключение к PostgreSQL (GORM)

```
internal/db/postgres.go
package db

import (
  "log"
  "os"
  "time"

  "gorm.io/driver/postgres"
  "gorm.io/gorm"
  "gorm.io/gorm/logger"
)

func Connect() *gorm.DB {
  // Упростим: возьмём DSN из переменной окружения DB_DSN
  // Пример: "host=127.0.0.1 user=postgres password=postgres dbname=pz6_gorm port=5432 sslmode=disable"
  dsn := os.Getenv("DB_DSN")
  if dsn == "" {
    log.Fatal("DB_DSN is empty")
  }

  db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
    Logger: logger.Default.LogMode(logger.Warn),
```

```
  })
  if err != nil {
    log.Fatal("connect db:", err)
  }

  // Настроим пул соединений
  sqlDB, err := db.DB()
  if err != nil {
    log.Fatal(err)
  }
  sqlDB.SetMaxOpenConns(20)
  sqlDB.SetMaxIdleConns(10)
  sqlDB.SetConnMaxLifetime(time.Hour)

  return db
}
```

Windows PowerShell (пример установки переменной):

```
setx DB_DSN "host=127.0.0.1 user=postgres password=postgres dbname=pz6_gorm port=5432 sslmode=disable"
```

macOS/Linux (bash/zsh):

```
export DB_DSN='host=127.0.0.1 user=postgres password=postgres dbname=pz6_gorm port=5432 sslmode=disable'
```

4) Модели и связи

- **User** — автор заметок (1 пользователь → много заметок).
- **Note** — заметка (принадлежит пользователю, имеет много тегов).
- **Tag** — тег (многие ко многим с заметками через таблицу `note_tags`).

`internal/models/models.go`

```
package models

import "time"

type User struct {
  ID        uint      `gorm:"primaryKey"`
  Name      string    `gorm:"size:100;not null"`
  Email     string    `gorm:"size:200;uniqueIndex;not null"`
  Notes     []Note    // 1:N
  CreatedAt time.Time
  UpdatedAt time.Time
}

type Note struct {
  ID        uint      `gorm:"primaryKey"`
```

```
  Title     string    `gorm:"size:200;not null"`
  Content   string    `gorm:"type:text"`
  UserID    uint      `gorm:"not null"`
  User      User
  Tags      []Tag     `gorm:"many2many:note_tags;"` // M:N
  CreatedAt time.Time
  UpdatedAt time.Time
}

type Tag struct {
  ID        uint      `gorm:"primaryKey"`
  Name      string    `gorm:"size:50;uniqueIndex;not null"`
  Notes     []Note    `gorm:"many2many:note_tags;"`
  CreatedAt time.Time
  UpdatedAt time.Time
}
```

5) Авто-миграции (создание таблиц)

cmd/server/main.go

```
package main

import (
  "log"
  "net/http"

  "example.com/pz6-gorm/internal/db"
  "example.com/pz6-gorm/internal/httpapi"
  "example.com/pz6-gorm/internal/models"
)

func main() {
  d := db.Connect()

  // Автоматически создаст (или обновит) таблицы под наши
модели
  if err := d.AutoMigrate(&models.User{}, &models.Note{},
&models.Tag{}); err != nil {
    log.Fatal("migrate:", err)
  }

  r := httpapi.BuildRouter(d)

  log.Println("listening on :8080")
  log.Fatal(http.ListenAndServe(":8080", r))
}
```

6) Роутер и хэндлеры (минимальный REST)

internal/http/router.go

```
package httpapi

import (
  "github.com/go-chi/chi/v5"
```

```
      "gorm.io/gorm"
     )

     func BuildRouter(d *gorm.DB) *chi.Mux {
      r := chi.NewRouter()
      h := NewHandlers(d)

      r.Get("/health", h.Health)

      // Пользователи (упрощённо)
      r.Post("/users", h.CreateUser)

      // Заметки
      r.Post("/notes", h.CreateNote)       // создаём заметку с
тегами
      r.Get("/notes/{id}", h.GetNoteByID) // получаем заметку с
автором и тегами

      return r
     }
     internal/http/handlers.go
     package httpapi

     import (
      "encoding/json"
      "net/http"
      "strconv"

      "github.com/go-chi/chi/v5"
      "gorm.io/gorm"

      "example.com/pz6-gorm/internal/models"
     )

     type Handlers struct{ db *gorm.DB }

     func   NewHandlers(db   *gorm.DB)   *Handlers   {   return
&Handlers{db: db} }

     func   (h   *Handlers)   Health(w   http.ResponseWriter,   r
*http.Request) {
      writeJSON(w,   http.StatusOK,   map[string]string{"status":
"ok"})
     }

     type createUserReq struct {
      Name  string `json:"name"`
      Email string `json:"email"`
     }

     func  (h  *Handlers)  CreateUser(w  http.ResponseWriter,  r
*http.Request) {
      var in createUserReq
```

```
      if err := json.NewDecoder(r.Body).Decode(&in); err != nil
|| in.Name == "" || in.Email == "" {
        writeErr(w, http.StatusBadRequest, "name and email are
required")
        return
      }
      u := models.User{Name: in.Name, Email: in.Email}
      if err := h.db.Create(&u).Error; err != nil {
        writeErr(w, http.StatusConflict, err.Error()) // возможен
конфликт по unique email
        return
      }
      writeJSON(w, http.StatusCreated, u)
    }

    type createNoteReq struct {
      Title   string   `json:"title"`
      Content string   `json:"content"`
      UserID  uint     `json:"userId"`
      Tags    []string `json:"tags"` // имена тегов
    }

    func (h *Handlers) CreateNote(w http.ResponseWriter, r
*http.Request) {
      var in createNoteReq
      if err := json.NewDecoder(r.Body).Decode(&in); err != nil
|| in.Title == "" || in.UserID == 0 {
        writeErr(w, http.StatusBadRequest, "title and userId are
required")
        return
      }

      // Находим/создаём теги
      var tags []models.Tag
      for _, name := range in.Tags {
        if name == "" { continue }
        t := models.Tag{Name: name}
        if err := h.db.FirstOrCreate(&t, models.Tag{Name:
name}).Error; err == nil {
            tags = append(tags, t)
        }
      }

      note := models.Note{
        Title:   in.Title,
        Content: in.Content,
        UserID:  in.UserID,
        Tags:    tags,
      }
      if err := h.db.Create(&note).Error; err != nil {
        writeErr(w, http.StatusBadRequest, err.Error())
        return
      }
```

```
      // Вернём с автором и тегами
      if err := h.db.Preload("User").Preload("Tags").First(&note,
note.ID).Error; err != nil {
        writeErr(w, http.StatusInternalServerError, err.Error())
        return
      }
      writeJSON(w, http.StatusCreated, note)
    }

    func (h *Handlers) GetNoteByID(w http.ResponseWriter, r
*http.Request) {
      idStr := chi.URLParam(r, "id")
      id, err := strconv.Atoi(idStr)
      if err != nil || id <= 0 {
        writeErr(w, http.StatusBadRequest, "bad id")
        return
      }
      var note models.Note
      if err := h.db.Preload("User").Preload("Tags").First(&note,
id).Error; err != nil {
        writeErr(w, http.StatusNotFound, "note not found")
        return
      }
      writeJSON(w, http.StatusOK, note)
    }

    // helpers (единый JSON-ответ)
    type jsonErr struct{ Error string `json:"error"` }

    func writeJSON(w http.ResponseWriter, code int, v any) {
      w.Header().Set("Content-Type",          "application/json;
charset=utf-8")
      w.WriteHeader(code)
      _ = json.NewEncoder(w).Encode(v)
    }
    func writeErr(w http.ResponseWriter, code int, msg string) {
      writeJSON(w, code, jsonErr{Error: msg})
    }
```

7) Запуск и проверка

Windows PowerShell

```
    $env:DB_DSN="host=127.0.0.1 user=postgres password=postgres
dbname=pz6_gorm port=5432 sslmode=disable"
    go run ./cmd/server
```

Проверяем:

```
    # здоровье
    curl http://localhost:8080/health

    # создаём пользователя
    curl -X POST http://localhost:8080/users ^
```

```
      -H "Content-Type: application/json" ^
      -d "{\"name\":\"Alice\",\"email\":\"alice@example.com\"}"

    # создаём заметку с тегами
    curl -X POST http://localhost:8080/notes ^
      -H "Content-Type: application/json" ^
      -d                                    "{\"title\":\"Первая
заметка\",\"content\":\"Текст...\",\"userId\":1,\"tags\":[\"go\"
,\"gorm\"]}"

    # получаем заметку с автором и тегами
    curl http://localhost:8080/notes/1
```

8) Частые ошибки и моментальные решения

- **`db_dsn empty`** — не установлена переменная окружения: задайте `DB_DSN`.
- **`duplicate key value violates unique constraint`** — дублируется email/имя тега: это ожидаемо — в модели стоит `uniqueIndex`.
- **«relation does not exist»** — забыли `AutoMigrate` или запускаете на пустой БД без миграции.
- **Неправильный порт/пароль** — проверьте `DB_DSN` (host/port/user/password/dbname).

Кто сдал — тот молодец (чек-лист)

- Проект запускается командой `go run ./cmd/server`.
- Таблицы создаются автоматически через `AutoMigrate`.
- Связи **1:N** (User→Notes) и **M:N** (Note↔Tags) работают; `Preload("User").Preload("Tags")` возвращает связанные данные.
- Ручки `/users`, `/notes`, `/notes/{id}` отвечают корректно.

Требования к отчёту

1. Краткое описание, зачем ORM и чем помог GORM (3–5 предложений).
2. Скриншоты:
   - `curl`/Postman: создание пользователя, создание заметки, получение заметки (с автором и тегами).
   - Фрагмент схемы БД (скриншот из `psql \d` или любой GUI).
3. Ссылка на репозиторий (минимум: `go.mod`, исходники, README с примером `DB_DSN`).
4. Пара строк о проблемах и как вы их решили.

Контрольные вопросы

1. Что такое ORM и зачем она нужна, если есть `database/sql`? Приведите 2–3 плюса и 1–2 минуса.
2. Как в GORM описать связи **1:N** и **M:N**? Какие теги на полях вы использовали?
3. Что делает `AutoMigrate`? В каких случаях его недостаточно?
4. Чем `Preload` отличается от обычного `Find/First`? Когда его применять?
5. Как в GORM обработать ошибку нарушения уникального индекса? Что вернёт БД/драйвер?